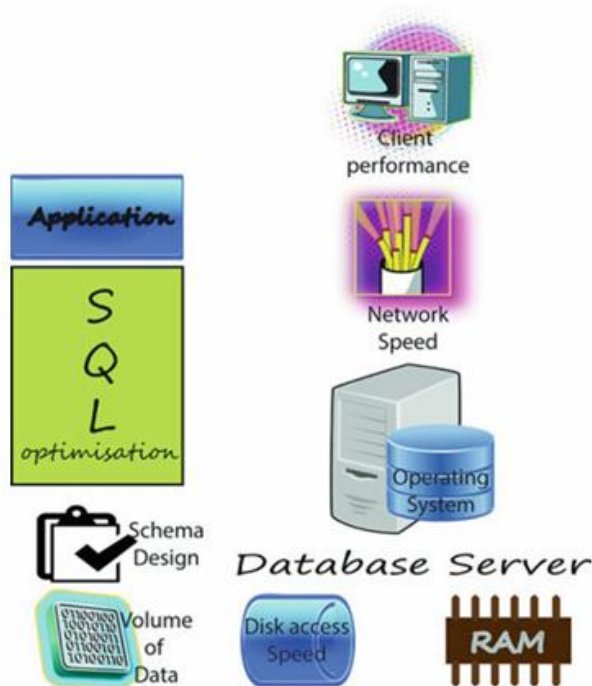


## Database Performance

Database performance refers to the speed, efficiency, and responsiveness of a database system while storing, retrieving, updating, and managing data. A high-performance database can process user requests quickly, support many users at the same time, and use system resources effectively. Good database performance is important for organizations because slow databases can delay business operations and reduce productivity. Many factors affect database performance, such as hardware quality, memory size, disk speed, network speed, query design, indexing, and database configuration. If the database server has enough RAM and fast storage devices like SSDs, performance improves significantly. Efficient SQL queries and proper indexing also reduce the time needed to access data. The Database Administrator (DBA) plays an important role in maintaining performance. The DBA monitors the system, tunes queries, manages storage, updates statistics, and ensures balanced resource usage. Techniques such as partitioning, caching, RAID, indexing, and SQL optimization are commonly used to improve performance. In modern organizations, database performance is essential because applications depend on fast and reliable data access. Poor performance can lead to delays, system failures, and user dissatisfaction. Therefore, continuous monitoring and tuning are necessary for maintaining an efficient database system.



**Fig. 14.1** Elements which influence database performance

### **14.3 Physical Storage**

Physical storage refers to the permanent storage devices used in a database system to store data safely for a long time. Unlike RAM, physical storage is non-volatile, which means data remains stored even when the power is turned off. Common storage devices include Hard Disk Drives (HDDs), Solid State Drives (SSDs), and flash memory. Since disk operations are slower than memory operations, database administrators try to reduce delays and improve data access speed.

#### **14.3.1 Block Size**

Block size is the amount of data read from or written to disk in one operation. The operating system manages data in blocks, and the database also uses database blocks for storage. Proper block size selection improves input/output performance and reduces wasted space. Smaller block sizes are useful in OLTP systems, while larger block sizes are useful in data warehouse systems that read large amounts of data.

#### **14.3.2 Disk Arrays and RAID**

Disk arrays and RAID (Redundant Array of Independent Disks) combine multiple disks to improve database storage performance and reliability. RAID can increase read and write speed by distributing data across several disks. It can also protect data if one disk fails. Different RAID levels provide different balances of speed, fault tolerance, and cost.

#### **14.3.3 Alternatives to HDD**

Traditional hard disk drives can be replaced by newer storage technologies such as SSDs, flash memory, and in-memory databases. Flash memory is faster than HDD in reading data because it has no moving parts. In-memory databases store data directly in RAM, giving very high speed performance. However, these alternatives are usually more expensive than HDD storage.

#### **14.3.4 Operating System (OS)**

The operating system is the software platform on which the database server runs. It manages hardware resources such as memory, CPU, and storage devices. Popular operating systems for databases include Windows, Linux, and Unix. A good operating system helps improve database stability, multitasking, and performance.

#### **14.3.5 Database Server Processes**

Database server processes are background tasks performed by the DBMS to support users and maintain system operations. These processes manage user connections, memory allocation, transaction processing, and recovery. They ensure that many users can access the database at the same time without problems. Efficient server processes improve database performance.

### **14.3.6 Archive Manager**

The archive manager is responsible for storing redo log files used for database recovery. These logs contain records of all database changes. If a failure occurs, archived logs help restore the database to a previous correct state. Although archiving improves data safety, it may slightly reduce performance because additional writing to disk is required.

### **14.3.7 Schema Level: Data Types, Location and Volumes**

Schema design decisions such as choosing data types, table structures, and storage methods affect database performance. Proper normalization reduces data duplication, while denormalization can improve query speed by reducing joins. Large data types like XML and CLOB require more storage and processing time. Therefore, careful schema design is important for efficient databases.

### **14.3.8 SQL Optimisation**

SQL optimization is the process of executing queries in the fastest and most efficient way. The DBMS optimizer checks different methods such as indexes, full table scans, and join orders before selecting the best query plan. It uses table statistics like size and data distribution. Good optimization reduces CPU usage and improves query response time.

### **14.3.9 Indexes**

Indexes are special database structures used to speed up data retrieval. They work like the index of a book, helping the system locate rows quickly without scanning the entire table. Indexes improve SELECT queries but may slow INSERT, UPDATE, and DELETE operations because indexes must also be updated.

#### **14.3.9.1 B-Tree Indexes**

B-Tree indexes are the most common type of index used in databases. They store keys in sorted tree structure, allowing fast searching, insertion, and deletion. They are useful for columns with many unique values such as employee ID or account number. B-Tree indexes are widely used in OLTP systems.

#### **14.3.9.2 Bitmap Indexes**

Bitmap indexes are designed for columns with low cardinality, meaning columns with few distinct values such as gender, yes/no, or status. They store data using bitmaps of 0s and 1s. Bitmap indexes are very efficient for complex analytical queries in data warehouses, but they are slower for frequent updates.

#### **14.3.9.3 Reverse Key Index**

Reverse key indexes reverse the order of index key values before storing them. This prevents too many inserts from going to the same disk block, reducing hot spots and

contention. They are useful in systems where sequential values are inserted continuously, such as invoice numbers or transaction IDs.

#### **14.3.9.4 Function-Based Indexes**

Function-based indexes are created on expressions or functions instead of normal columns. For example, an index can be created on UPPER(name) to support case-insensitive searches. These indexes improve query speed when functions are used frequently in WHERE clauses.

#### **14.3.9.5 Final Thoughts on Indexes**

Indexes should be reviewed regularly because unnecessary indexes increase storage and slow data modification operations. As data size and usage patterns change, some indexes may become useful while others may become unnecessary. Proper index maintenance is essential for good performance.

#### **14.3.10 Network**

The network connects database clients to the server. Slow network speed can delay query results, especially when users are located far from the server. Database administrators can improve performance by tuning network packet sizes and reducing unnecessary data transfer.

#### **14.3.11 Application Design with Performance in Mind**

Applications should be designed carefully to use efficient SQL queries and proper database structures. Poorly designed applications may create excessive joins, too many indexes, or unnecessary data requests. Involving database administrators during development helps improve system performance.

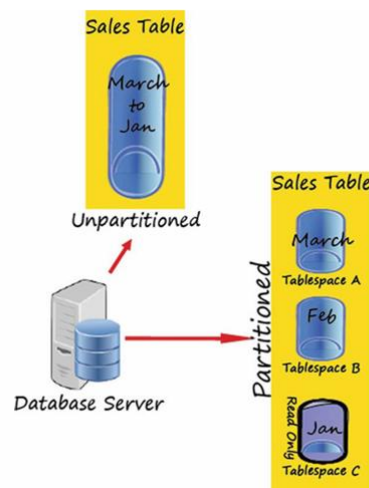
#### **Indexed Organised Table (IOT)**

An Indexed Organised Table stores table data directly inside the primary key index structure instead of separate table storage. This gives faster access when data is frequently searched using the primary key. However, secondary indexes become more difficult to maintain.

#### **Partitioning**

Partitioning divides a very large table into smaller parts called partitions based on rules such as date range or location. It improves query speed because only relevant partitions are searched. Partitioning also makes backup, maintenance, and data management easier.

**Fig. 14.5** Partitioning data across disks



**Database Tuning** – Database tuning is the process of improving the performance and efficiency of a database system by monitoring, analyzing, and optimizing its components. It involves identifying performance problems and applying suitable changes in hardware, database configuration, SQL queries, and application design to make data processing faster and more efficient.

In simple terms, database tuning means adjusting and optimizing a database system so that it uses resources like CPU, memory, storage, and network in the best possible way. The main goal is to reduce response time and improve the speed of data retrieval and transactions.

Database tuning is an ongoing process that includes monitoring system performance, analyzing bottlenecks, and applying corrective actions such as adding hardware resources, modifying SQL queries, changing database parameters, or improving application logic. After applying changes, performance is again monitored to check improvement, making it a continuous cycle.